



DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

MACHINE LEARNING FOR ROBOTICS I

Report

Complete report of all the lab assignments

Author:

Chiara Masaniello

Student ID:

s8434670

Professor:

Stefano Rovetto

2025/2026

Contents

1	Introduction	2
2	Lab 1: Naïve Bayes Classifier	3
2.1	Lab method	3
2.1.1	Data preprocessing	3
2.1.2	Build Naïve Bayes classifier	3
2.1.3	Laplace smoothing	4
2.2	Results	4
2.3	Conclusion	5
3	Lab 2: k-Nearest Neighbors classifier	6
3.1	Lab method	6
3.1.1	Obtain the dataset	6
3.1.2	Build the kNN classifier	6
3.1.3	Test the classifier	6
3.2	Results	7
3.3	Conclusions	8
4	Lab 3: Shallow neural networks	10
4.1	Lab method	10
4.1.1	Obtain the dataset	10
4.1.2	Build the shallow neural network	10
4.1.3	Select the model	10
4.2	Results	11
4.3	Conclusions	12
5	Lab 4: Basic autoencoder	14
5.1	Lab method	14
5.1.1	Obtain and prepare data	14
5.1.2	Build the autoencoder network	14
5.1.3	Train the autoencoder network	15
5.2	Results and Conclusion	15
5.2.1	Task2: build the autoencoder	15
5.2.2	Task3: Image reconstruction	16
5.3	Overall Conclusions	17

1 Introduction

The report collects all the labs done during the course of Machine Learning, it examines the application of various machine learning techniques through a series of practical labs. The topics covered are:

- **Lab 1: Naïve Bayes classifier**
with the implementation of a weather and breast cancer datasets, it was trained and tested the Naïve Bayes Classifier. The scope of this lab was to demonstrate the capability in pattern recognition of this classifier;
- **Lab 2: k-Nearest Neighbors (kNN) classifier**
through the `keras` libraries and the mnist dataset it aims to analyze the distances between class impact to the choice of k in the metric of risk;
- **Lab 3: Shallow neural networks**
through the linear regression and concepts as generalization, cross-validation and the choice of the shallow neural networks it was analyze the "Condition Based Maintenance of Naval Propulsion Plants" to understand the compromise between the learn ability of the network and the overfitting risk;
- **Lab 4: Basic autoencoder**
it demonstrate through an autoencoder how a neural network can learn compact representations of data in an unsupervised manner, and how to evaluate the quality of the mnist dataset reconstruction.

The report follows this structure, it shows for every lab the theory concepts, the implementation choices and the conclusion as a results of the applications.

2 Lab 1: Naïve Bayes Classifier

The **Bayes classifier** is used for decision problems based on recognizing the state of nature, which represents the set of all possible outcomes or events that could occur in a particular situation.

The features of state of nature are the variables that represent the state itself. These features can be continuous or discrete; in this **Lab 1** we focus on *discrete* variables.

To simplify the problem used for the classification task, the **Naïve Bayes** classifier has been implemented based on the *Bayes' theorem* with the assumption that all the features are independent of each other.

2.1 Lab method

This laboratory aims to demonstrate the validity of the **Naïve Bayes** classifier through three steps: data pre-processing, classifier implementation, and the introduction of Laplace smoothing.

2.1.1 Data preprocessing

The first task represents the preparation for building the classifier.

Through the `pandas` library, both datasets are imported: *weather* and *breast cancer*. The datasets are split into features X and labels y , and they are cleaned by an additional check that verifies the presence of missing data. If missing data are less than the 15%, it is imputed using the `SimpleImputer(strategy='most_frequent')`; otherwise, rows containing missing values for that feature are removed.

The dataset is split into 75% training and 25% testing using the function `train_test_split()` which ensures that the class distribution is preserved across the two subsets. The *train set* is the subset of data used to fitting and predict the model, it analyzes the data and it used from the classifier to learn. The **test set** is the subset which be used to testing the model to give an accurate evaluation of the final model fit.

For each feature, the set of possible categorical *levels* is extracted from the training set and stored in the dictionary `levels`. This structure is later passed to the classifier through the `fit()` method to detect unseen values during prediction.

2.1.2 Build Naïve Bayes classifier

The classifier consists of training, predicting and testing the data.

- **Training**

During training, the classifier computes the class priors using the function `calculate_class_priors()` which calculates the prior probability as:

$$P(c_i) = \frac{\text{occurrences of class } c_i}{\text{total number of observations}}$$

This is used to compute the conditional probabilities of each feature for each class. For this purpose, the `crosstab` function from `panda` was introduced, with which it is possible to calculate the likelihood representing the probability of observing feature x_i given class c_i . These conditional probabilities for each feature were implemented by the function

`calculate_feature_class_likelihood()` through *Bayes' theorem* formula:

$$P(c_i|x) = \frac{P(c_i) \cdot P(x_i|c_i)}{P(x)}$$

where

- $P(c_i|x)$ is the posterior probability of class c_i given the feature $\mathbf{x}$;
- $P(c_i)$ is the prior probability of class c_i ;
- $P(x_i|c_i)$ is the likelihood of feature x_i given class c_i , obtained from the training set;
- $P(\mathbf{x})$ is the probability of the feature $\mathbf{x}$ to occur, regardless the class.

To avoid numerical underflow due to the product of many small probabilities, the implementation uses logarithms.

- **Predict**

For each test sample, the classifier assigns a score through the formula:

$$\log P(c) + \sum_j \log P(x_j|c)$$

Before scoring, the method checks whether each feature value is present in the known levels; in this case, depending on the parameter `unknown`, the sample is either discarded or an error is raised.

- **Test**

The last step is the most significant part to prove the accuracy of the classifier. It takes the features of the testing split of the dataset and uses the previous functions to predict the labels. The difference between the predicted value and the real one determines the accuracy of the classifier.

With this classifier, we are able to answer questions such as *"With this weather, is it possible to play tennis?"* and *"Is this woman subject to having cancer?"*.

2.1.3 Laplace smoothing

When splitting the dataset, there is the possibility of encountering a label in the test data that was not present in the train data.

In the current classifier, in such a case the probability associated with this occurrence is zero, which can lead to inaccurate predictions since a single zero makes the entire product zero.

To handle the *zero probability problem*, Laplace additive smoothing was introduced. We can distinguish three cases considering this smoothing element α :

- **no smoothing** ($\alpha = 0$): when the test observes a new feature, it will associate with zero likelihood, leading to *overfitting*;
- **moderate smoothing** ($\alpha = 1$): a pseudo-count is added to all the feature occurrences through the formula that computes the probability of observing the feature i :

$$P(x = i) = \frac{n_i + \alpha}{N + \alpha v}$$

where

- n_i is the number of occurrences of feature value i in class c ;
- α is the smoothing parameter;
- N is the total number of observations in class c in the training set;
- v is the number of different values for that feature.

This helps the classifier to generalize beyond the observed feature.

- **strong smoothing** ($\alpha \gg 1$): if α is too large, there is the possibility that the probabilities become the same for all classes, leading to *underfitting*.

To avoid the problem of encountering unseen combinations of features and classes during classification, a moderate smoothing of $\alpha = 1$ was introduced. It redistributes the probability mass from observed feature values to unseen ones, reducing bias and avoiding zero probabilities.

2.2 Results

The experimental results highlight how the performance of the Naïve Bayes classifier strongly depends on the size and quality of the available data.

- **WEATHER DATASET**

size: 14 observations, 4 categorical features, 2 classes

missing data handling: none

accuracy: 0.5 Due to the extremely small size of the dataset, the obtained accuracy is highly unstable and sensitive to the train/test split. With a 75/25 split, only 10 samples are used for training and 4 for testing, making the evaluation unreliable from a statistical perspective.

- **BREAST CANCER DATASET**

size: 286 observations, 9 categorical features, 2 classes

missing data handling: only 2 features contain missing value (approximately –5% of observations)

accuracy: 0.75 The larger size of this dataset allows for a more reliable evaluation of the classifier. The achieved accuracy of 0.75 demonstrates that the Naïve Bayes model performs reasonably well when sufficient training data are available.

Considering the presence of unseen feature combinations in the test set, the effectiveness of Laplace smoothing can be observed in **Table 3** that reports the comparison between the classifier without smoothing and with Laplace smoothing.

α	Accuracy
0	0.735
1	0.750

Table 1: Accuracy comparison with and without Laplace smoothing

Although the improvement in accuracy is limited, this behavior is expected given the relatively small number of discarded samples (only one out of 68 in the test set).

2.3 Conclusion

The results also confirm that the **Naïve Bayes** classifier is highly sensitive to the train–test split when the dataset is small. This is evident in the Weather dataset, instead the Breast Cancer dataset provides more stable and meaningful results.

The Naïve hypothesis of the independence of all the labels is a limit of the classifier, it is not suitable for real-world models. Another limitation can be seen in the preprocessing, if preprocessing is poor and missing-data handling is ineffective the performance will suffer.

As shown in **Figure 8** and **Figure 10**, the stratified split preserves the original class distribution across training and test sets, avoiding the introduction of bias during evaluation.

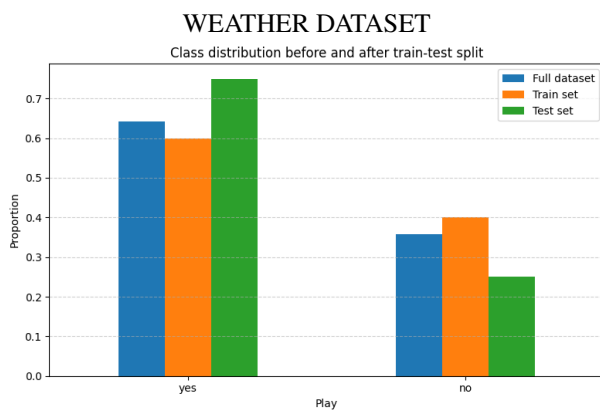


Figure 1: Importance of the split for the *weather* dataset

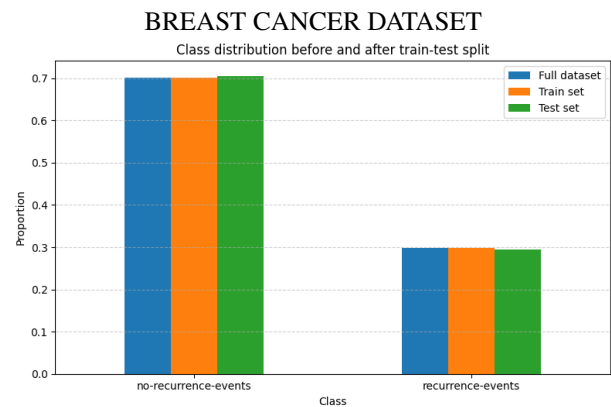


Figure 2: Importance of the split for the *breast cancer* dataset

Although this laboratory focuses on categorical variables, continuous features can also be handled by the classifier by modeling probability density functions (e.g., Gaussian distributions) instead of relying on frequency counts.

Overall, this laboratory illustrates both the strengths and the limitations of the Naïve Bayes classifier: it is simple, fast, and effective on categorical data, but its independence assumption and sensitivity to data scarcity highlight the importance of proper preprocessing and dataset size in practical applications.

3 Lab 2: k-Nearest Neighbors classifier

The **K-Nearest Neighbors (KNN)** is a supervised classifier that uses the proximity between points to resolve *classification problems*. The algorithm classifies a test sample by finding the K nearest neighbors in the feature space and assigning the most common class among them through majority voting.

3.1 Lab method

This laboratory aims to demonstrate the validity of the **kNN** classifier through three steps: obtaining the dataset, classifier implementation, and classifier testing.

3.1.1 Obtain the dataset

In this laboratory, the *mnist* and *wine* datasets are obtained through the Python library `scikit-learn`. Both distinguish between input features (the measurements used to make predictions) and the target (the predicted value). We separate the features X from the target y .

For the *mnist* dataset, each 28×28 image is reshaped into a 784-dimensional vector and normalized by dividing pixel intensities by 255. This preprocessing step is implemented inside the `mnist_subsets()` function, which also extracts random subsets of fixed size for training and testing, as required by the assignment.

For the *wine* dataset, feature scaling is performed using `MinMaxScaler()` function, ensuring that all continuous variables lie in the interval $[0, 1]$, as recommended for distance-based methods.

Compared to **Lab 1**, data preprocessing and cleaning are not necessary because this laboratory considers 'toy' problems, so the datasets are already ready for building the classifier.

3.1.2 Build the kNN classifier

Unlike **Naïve Bayes**, which relies on prior distributions, **kNN** classifier structure is entirely determined from data. However the *complexity* of the model depends on the number of stored samples.

It is necessary to store the data in the training set through the `fit()` method. In this step the model does not learn parameters; it simply stores the training samples.

To perform classification, a `predict` method is implemented. It computes the classification for each test sample through the following steps:

- **Distance Calculation**

This method computes *Euclidean distance* using the `np.linalg.norm()` function, which implements the formula that calculates the distance between two points x_i and x_j across d features:

$$d(x_i, x_j) = \sqrt{\sum_{k=1}^d (x_{ik} - x_{jk})^2}$$

- **Neighbor Selection**

The K training samples with smallest distances are identified through the `np.argsort()` function. The choice of k significantly affects the behavior of the model; therefore, different values of k are tested and their accuracy compared.

- **Majority Voting**

This determines the predicted class using the `np.bincount()` and `np.argmax()` functions, which count the class labels among the K neighbors and predict the most frequent class.

3.1.3 Test the classifier

The `test` method computes classification accuracy as the fraction of correct predictions.

For the *mnist* dataset (the larger one), the function `mnist_subsets()` is used to generate 6 training subsets of 10,000 samples and 5 test subsets of 2,000 samples. For each value of k, the classifier is trained and tested on randomly selected subsets, and the accuracy is averaged over 5 repetitions; this is necessary to reduce

computation time.

For each class, all accuracies are compared using the `accuracy_score()` function and the results are stored in the dictionary `class accuracies`.

To analyze the impact of K (number of neighbors), the model is trained and tested multiple times with $k \in \{1, 2, 3, 4, 5, 10, 15, 20, 30, 40, 50\}$. The test is computed with different random subsets (for *mnist*) or the same split (for *wine*), and the mean accuracy (`mean_accuracy()` function) and standard deviation (`std_accuracy()` function) are computed to assess stability.

3.2 Results

Two datasets are used to evaluate the classifier:

- **WINE DATASET**

size: 178 samples, 13 continuous features, 3 classes. The moderate-sized dataset is used to analyze different K parameters;

split: 70% training (124 samples), 30% testing (54 samples);

preprocessing: `MinMaxScaler()` to normalize features to $[0, 1]$ range.

k	mean_accuracy	std_accuracy
1	0.9630	0.0000
2	0.9444	0.0000
3	0.9630	0.0000
4	0.9630	0.0000
5	0.9444	0.0000
10	0.9630	0.0000
15	0.9815	0.0000
20	0.9815	0.0000
30	0.9630	0.0000
40	0.9630	0.0000
50	0.9630	0.0000

Table 2: k-value and accuracy comparison for the *wine* dataset

In this case, the database is small and well separated into classes, which is reflected in the results: the accuracy is high (0.94 - 0.98) and the standard deviation is zero.

- **MNIST DATASET**

size: 60,000 training images, 10,000 test images. High-dimensional dataset is used to test scalability;

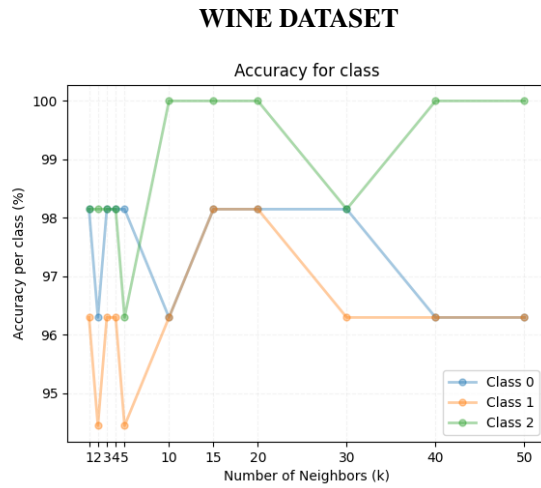
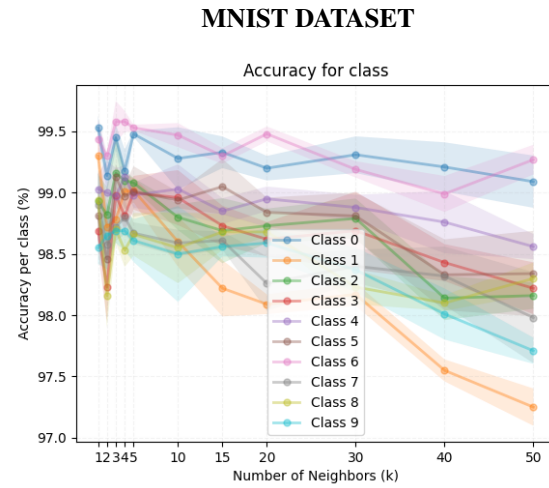
split: introduction of subsets, each with 10,000 training samples and 2,000 test samples;

preprocessing: flatten 28×28 images to 784-dimensional vectors, normalize to $[0, 1]$.

k	mean_accuracy	std_accuracy
1	0.9505	0.0024
2	0.9389	0.0008
3	0.9509	0.0033
4	0.9436	0.0047
5	0.9508	0.0014
10	0.9482	0.0040
15	0.9425	0.0006
20	0.9311	0.0028
30	0.9261	0.0028
40	0.9230	0.0030
50	0.9178	0.0070

Table 3: k-value and accuracy comparison for the *mnist* dataset

In this case, the accuracy decreases as k increases. With larger databases where the data are not as well separated into classes, the accuracy remains high but shows more variation. The standard deviation is near zero but larger than in the wine dataset.

Figure 3: Accuracy on *wine* datasetFigure 4: Accuracy on *mnist* dataset

3.3 Conclusions

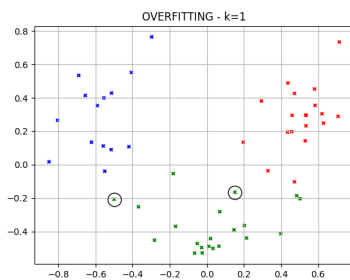
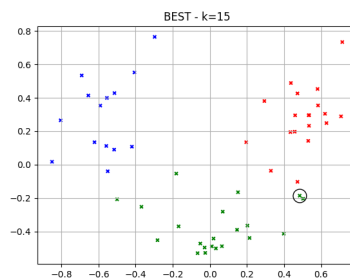
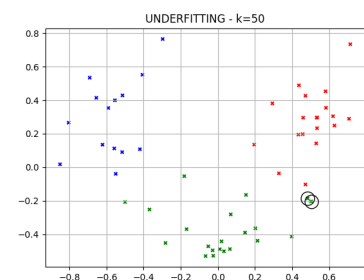
The different database sizes demonstrate how dimension and splitting influence the classifier, but the standard deviation is small in both cases, so performance is better with smaller databases, but the model is still stable even with larger ones.

The influence of k on model performance depends on its value:

- **smaller values ($k \in \{1, 2\}$):** the accuracy is very high, but the model captures data noise, leading to overfitting. The decision boundaries are highly irregular and sensitive to individual training samples;
- **medium values ($k \in \{3, 4, 5, 10, 15\}$):** these values represent the best compromise between accuracy and noise reduction. They achieve optimal generalization by smoothing local irregularities while preserving important class boundaries;
- **higher values ($k \in \{20, 30, 40, 50\}$):** these k -values lead to underfitting. The decision boundaries become overly smooth, and the model loses the ability to capture local patterns, resulting in decreased accuracy.

The most significant values of k for both datasets can be observed in the PCA visualizations:

• WINE DATASET

Figure 5: $k = 1$ - OVERFITTING
misclassifications but high sensitivityFigure 6: $k = 15$ - BEST K
optimal balanceFigure 7: $k = 50$ - UNDERFITTING
smoother but less accurate boundaries

- MNIST DATASET

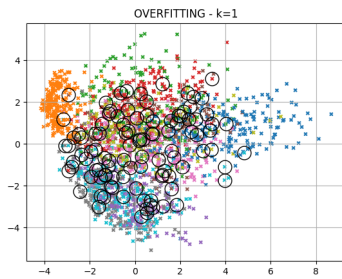


Figure 8: k = 1 - OVERFITTING
sensitivity to noise in pixel data

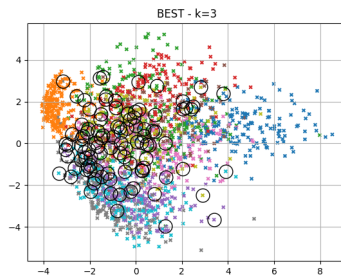


Figure 9: k = 3 - BEST K
best performance and noise balance

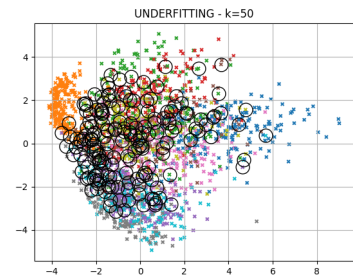


Figure 10: k = 50 - UNDERFITTING
degraded performance

The K-Nearest Neighbors classifier demonstrates strong performance on both moderate and high-dimensional datasets when appropriately tuned. The implementation successfully handles multi-class classification and provides consistent results across multiple test iterations.

Compared to the **Naiïve Bayes** classifier has a significant limitation: the computation of distances slows down prediction, especially with large datasets. Each prediction requires calculating distances to all training samples, resulting in $O(n \times d)$ complexity per test sample, where n is the training set size and d is the number of features.

Another limitation is the sensitivity to feature scaling. The model requires normalization because the Euclidean distance is affected by the magnitude of features. Features with larger ranges would dominate the distance calculation without proper scaling.

Despite these limitations, this classifier is very versatile. It also works for *regression problems*, in which case the output would be the average of the neighbors' values instead of the mode (majority vote). This flexibility makes **kNN** applicable to a wide range of machine learning tasks.

4 Lab 3: Shallow neural networks

A **shallow neural network** is a feedforward artificial neural network with a single hidden layer between the input and output layers. It provides a simpler architecture that can still approximate complex non-linear functions.

This laboratory focuses on *regression tasks*, where the network learns to map continuous input features to continuous output values by minimizing the mean squared error (MSE) between predictions and true targets.

4.1 Lab method

This laboratory demonstrates the implementation and systematic evaluation of shallow neural networks through three main tasks: obtaining the dataset, neural network regression implementation, and model selecting via cross-validation.

4.1.1 Obtain the dataset

The first task establishes the foundation for neural network training through proper data preparation.

The dataset used is the *Condition Based Maintenance of Naval Propulsion Plants* from the [UCI Machine Learning Repository](#). The task is to predict two quantities expressing the health status of a ship gas turbine based on 16 sensor measurements.

The dataset is divided and shuffled into features X (sensor measurements) and targets y (turbine health indicators). This makes it a multi-output regression problem with 16 inputs and 2 outputs.

Train data are normalized using the `minmax_normalization_train()` functions that used the formulas:

$$X_{norm} = \frac{X_{train} - X_{min}}{X_{max} - X_{min}} \quad y_{norm} = \frac{y_{train} - y_{min}}{y_{max} - y_{min}}$$

this transformation maps all values to the range $[0, 1]$, which is essential for neural network training because prevents features with large magnitudes from dominating the learning process and also matches the output range of sigmoid activation functions.

4.1.2 Build the shallow neural network

Before building the network, it is necessary to divide the dataset. Compared to the two division of the previous laboratories, dataset is divided into three subsets: training set (70% of data used to optimize network weights), validation set (15% of data used to monitor overfitting during training) and test set (15% of data held out for final performance evaluation).

The training set updates weights, the validation set provides feedback during training to detect overfitting, and the test set provides an unbiased estimate of generalization performance on completely unseen data.

The architecture of the shallow neural network is simple:

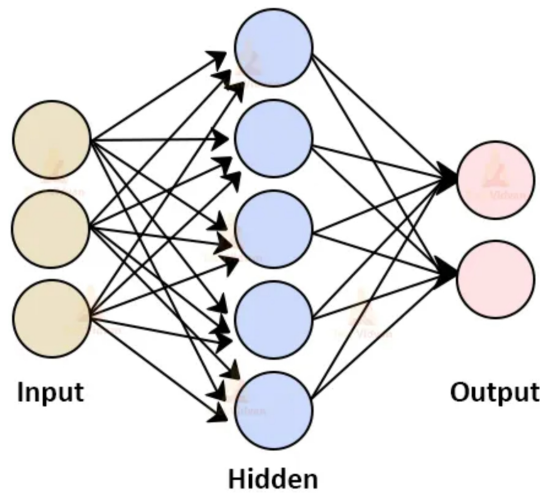
The *input layer* is composed on d neurons (number of input features), the *hidden layer* is composed on h neurons (number of hidden neurons) and it works with the `sigmoid` activation function which enables the network to learn complex patterns and mapping inputs to $(0,1)$, instead *output layer* is composed on c neurons (number of output dimensions) and it works with **linear** activation function which allows the network to produce any real-valued output, which is necessary for *regression tasks* where targets can take any continuous value.

The final error of *regression task* that approximates a function $y(t)$ which maps input x in target t , it is the squared error (MSE) that measures the average squared difference between predictions and true values.

The optimization uses the `Adam` Python optimizer through 30 epochs (iterations through the entire training set).

4.1.3 Select the model

The best performance model is selected based on the lowest test MSE and the hyperparameters (h selection) via cross-validation.

Figure 11: Architecture of **shallow neural network**

To accomplish this scope the test is separated into 5 equal folds, for each fold k one of this set will be considered as the 'real' tester, the other four will be used as training set. Instead, from the training set will be reserved through the `validation_split()` Keras function the 10% for validation.

This approach differs from **Task 2**: instead of a fixed 70/15/15 split, cross-validation uses all data more efficiently by rotating which portion serves as the test set.

The test is repeated 5 times for each $h \in \{4, 8, 12, 16, 20, 40\}$, it select the train and test subsets randomly and select the fold considering:

- **median**: lowest median test MSE across folds;
- **interquartile range**: smaller ratio computes as

$$\frac{P75 - P25}{median}$$

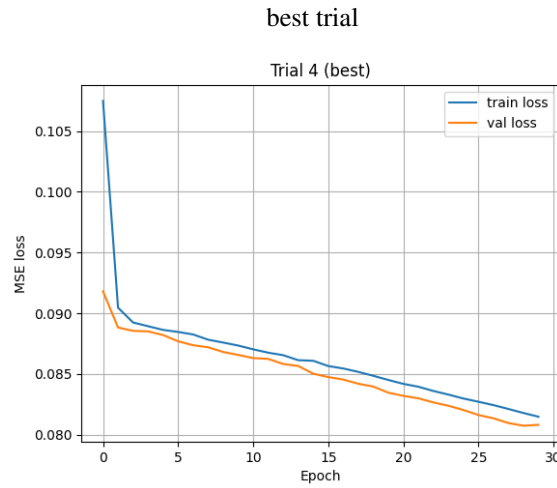
The h values influenced the capacity of the network. Small h values produce simpler models with higher bias and lower variance, this can bring to underfitting because the is the risk the model will not understand data. Instead with a big h the model will be more complexity with more variance but a little bias, which can lead to overfitting, the model learn well the data but this can change with the training subsets.

4.2 Results

The build shallow network shows the following results:

- **TEST**
best MSE = $7.9410e - 02$
- **TRIAL**
mean MSE = $8.2821e - 02$
standard deviation MSE = $2.0575e - 03$

This results are faithful to the best trial, shows below.

Figure 12: best h - worst trial

For this model it is possible analyze the h folds:

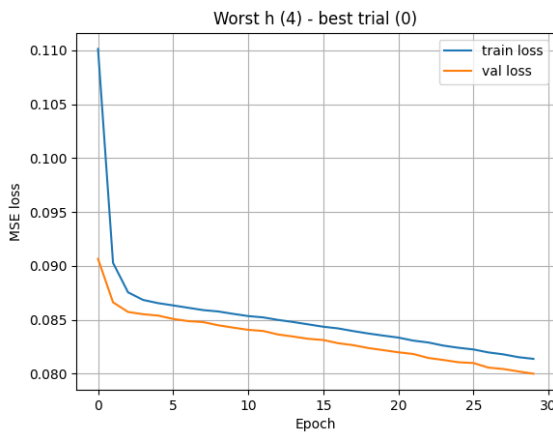
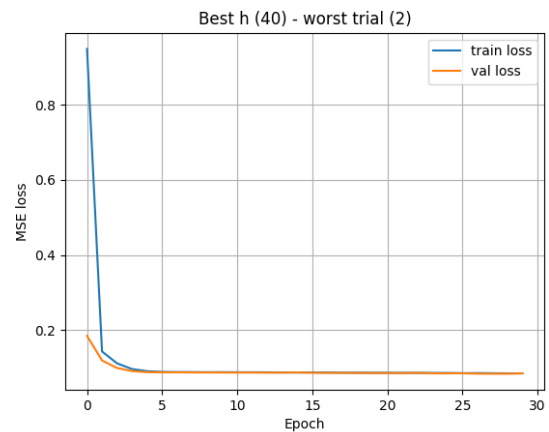
h	median	p25	p75	ratio
4	8.525×10^{-2}	8.423×10^{-2}	8.630×10^{-2}	2.429×10^{-2}
8	8.234×10^{-2}	8.136×10^{-2}	8.260×10^{-2}	1.510×10^{-2}
12	8.277×10^{-2}	7.997×10^{-2}	8.333×10^{-2}	4.059×10^{-2}
16	8.061×10^{-2}	7.978×10^{-2}	8.224×10^{-2}	3.050×10^{-2}
20	8.097×10^{-2}	8.085×10^{-2}	8.250×10^{-2}	2.035×10^{-2}
40	7.836×10^{-2}	7.693×10^{-2}	8.013×10^{-2}	4.085×10^{-2}

Table 4: h -folds

4.3 Conclusions

The cross-validation results clearly highlight the bias–variance trade-off associated with the choice of the hidden layer size h .

Small networks (e.g., $h = 4$) show consistently higher MSE, indicating underfitting due to insufficient model capacity. Large networks (e.g., $h = 40$) achieve the lowest median MSE (7.836×10^{-2}), confirming their higher expressive power, but also exhibit the largest variability across folds, making them more sensitive to weight initialization and therefore less stable.

Figure 13: worst $h = 4$ - best trialFigure 14: best $h = 40$ - worst trial

Intermediate values such as $h = 8$ and $h = 16$ provide a more balanced compromise.

In particular:

- $h = 8$: achieves the best *stability* (lowest ratio), although with slightly higher error.
- $h = 16$: offers the best *balance* between accuracy and robustness (good median, moderate ratio).

Depending on whether accuracy or stability is prioritized, either $h = 40$ (best performance) or $h = 16$ (best compromise) can be selected.

Overall, the shallow neural network successfully performs multi-output regression, and the laboratory demonstrates the importance of cross-validation and multi-start optimization when tuning neural network hyperparameters.

5 Lab 4: Basic autoencoder

An **autoencoder** is a special type of neural network designed to learn efficient data representations in an unsupervised manner. The network is trained to *reconstruct* its input through a bottleneck layer, forcing it to learn a compressed representation that captures the most salient features of the data.

Autoencoders consist of two main components: an **encoder** that compresses the input into a lower-dimensional latent space, and a **decoder** that reconstructs the original input from this compressed representation. This architecture enables applications ranging from dimensionality reduction and feature learning to denoising and image inpainting.

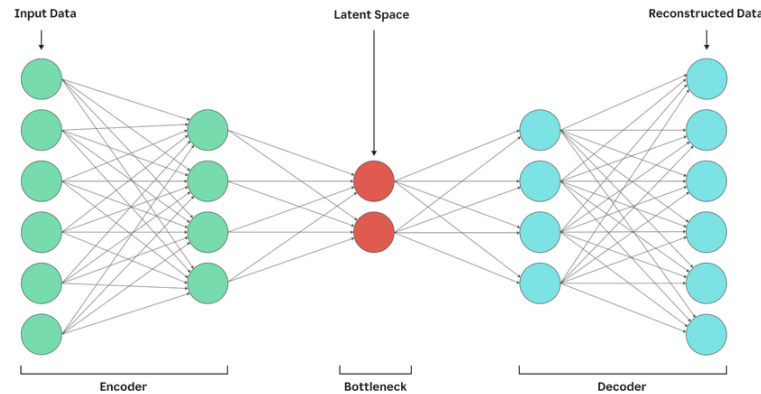


Figure 15: Autoencoder structure

This laboratory explores two fundamental applications of autoencoders: creating low-dimensional embeddings for visualization (Task 2) and learning to reconstruct masked images (Task 3).

5.1 Lab method

This laboratory demonstrates the versatility of autoencoders through three progressive tasks: data preparation, building autoencoder network, and autoencoder training.

5.1.1 Obtain and prepare data

The first task establishes the data processing pipeline for working with the `MNIST` handwritten digit dataset. The `MNIST` dataset is imported using `TensorFlow` Python library. It is split in training set (60,000 grayscale images of handwritten digits, from 0 to 9) and test set (10,000 grayscale images). The image dimension is 28×28 pixels with values in range $[0, 255]$.

To analyze the ability of the autoencoder to separate different digit classes, the implementation allows parametric selection of two classes from the ten available digits. The filtering is implemented through the `filter_classes()` function that use a boolean mask by `isin` function.

Before working with the images, they need to be normalized and flattened. The `normalize_and_flatten()` function scaling the pixel values from $[0, 255]$ to $[0, 1]$, and reshaping the 2D arrays (28×28) to 1D vectors: $X \in \mathbb{R}^{N \times 28 \times 28} \rightarrow X \in \mathbb{R}^{N \times 784}$.

5.1.2 Build the autoencoder network

The minimal autoencoder consists of only 2 hidden units to project high-dimensional digit images onto a 2D plane for visualization, to learn the most discriminative features the model need 2 neurons.

The shallow autoencoder has a symmetric encoder-decoder structure

- **encoder:** it is used for dimensionality reduction

Input Layer (flattened image 28×28) $\rightarrow$ Hidden Layer (2 units - sigmoid activation)

- **decoder:** it is used for image reconstruction

Hidden Layer (2 unit) → Output Layer (784 units - linear activation)

The autoencoder learns to minimize reconstruction error by training the training subset using the Adam optimizer, iterated on 80 epochs and using 256 samples on batch size. The 10% of training data will be used as validation set.

The training consist of 5 independent models, for each it is computed the MSE, the model will be selected considering the lower MSE with a target threshold 0.07 to determined an acceptable reconstruction quality. If the target is not reached after maximum restarts, the best attempt is used with a warning.

After training the full **autoencoder**, only the **encoder** portion is used for inference, each image is represented by 2 coordinates (hidden unit activations) used for the plot in the 2D plane.

5.1.3 Train the autoencoder network

This task demonstrate the ability of the **autoencoder** to reconstruct missing or corrupted portions of images through *image inpainting*

Unlike the 2 units structure build in **Task 2**, it is implemented a deeper architecture with 3 hidden layers, which has greater capacity to handle complex reconstruction. The **encoder** implemented 3 linear function ReLU (respectively vectors 256, 128 and 64) with the last one represents the bottleneck layers. Instead the **decoder** implemented two lineare function ReLU (with 128 and 256 unit vectors) and the output layer with a Sigmoid function with 784 units. This increased depth and width (64 bottleneck units against the 2 before) enables the network to preserve more information necessary for high-quality reconstruction.

To simulate different occlusion patterns, three types of masks are implemented. Considering the area to mask:

$$A = severity \times (H \times W)$$

- **horizontal stripes:** mask_{mode} = 'h', randomly masks full-width rows. Height of masked region:

$$h = \frac{A}{W}$$

- **vertical stripes :** mask_{mode} = 'v', randomly masks full-height columns. Width of masked region:

$$w = \frac{A}{H}$$

- **rectangular patches:** mask_{mode} = 'p', randomly masks square or rectangular regions. Dimensions satisfy

$$h \times w = A$$

The mask type is selected through mask_{mode} = 'mix', which randomly chooses one of the three patterns for each image, providing diverse training data. This prevents the model from overfitting to a specific occlusion pattern and ensures robust generalization.

To analyze how masking severity influences performance, the test is repeated for $s \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7\}$, representing 10% to 70% of pixels masked. Each severity level trains an independent autoencoder, allowing us to determine the threshold beyond which reconstruction becomes unreliable.

5.2 Results and Conclusion

5.2.1 Task2: build the autoencoder

The images below show the difference in performance between well-separated classes (e.g., (5, 7)) and similar classes (e.g., (3, 5)).

With **separate classes**, the **encoder** recognized very well the feature that distinguishes between the two digits. Instead the model is not good enough with the similar cases.

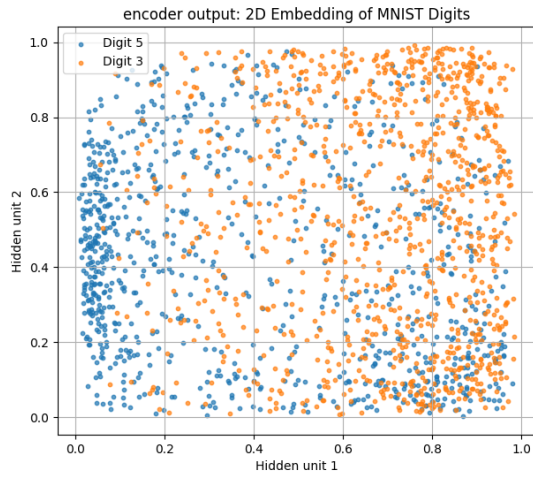


Figure 16: Similar classes (3,5)

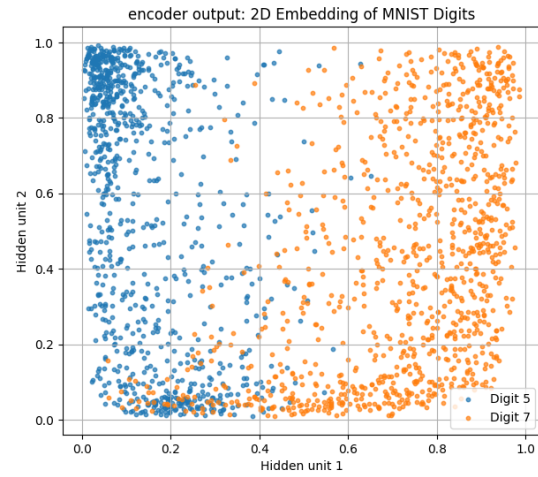


Figure 17: Separate classes (5,7)

The 2D embedding reveals distinct, non-overlapping clusters for the **distinct classes** (5, 7) one hidden unit likely encodes the *curvature* of digit 5, the other hidden unit encodes the *verticality* of digit 7.

In contrast, the model struggles with **similar classes** (3, 5). The plot shows significant overlap without clear separation. The visual similarities between digits 3 and 5 are too subtle to be captured with only 2 hidden units.

The shallow autoencoder effectively separates visually distinct digits but cannot distinguish similar ones with such limited representational capacity.

5.2.2 Task3: Image reconstruction

The **autoencoder** model is selected based on the lowest test MSE for each severity level. The results are summarized in table below:

Severity	Test MSE	Quality
0.10	0.0110	Excellent
0.20	0.0168	Excellent
0.30	0.0227	Good
0.40	0.0298	Good
0.50	0.0376	Good
0.60	0.0451	Okay
0.70	0.0539	Okay

Table 5: Comparative table of the **basic autoencoder**

In the images below show the difference in reconstruction quality:

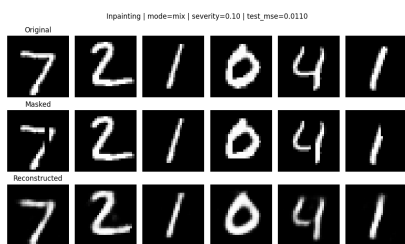


Figure 18: severity = 0.10

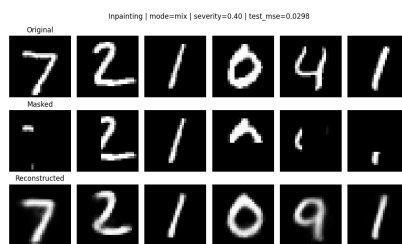


Figure 19: severity = 0.40

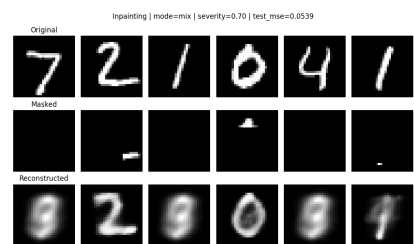


Figure 20: severity = 0.70

The **autoencoder** maintains good reconstruction capacity until approximately 50% masking.

With lower severity ($s \in \{0.10, 0.30, 0.50\}$) the MSE remains low (0.01-0.04) and reconstructions are visually

consistent and recognizable. The MSE increases approximately linearly with severity, and reconstruction quality degrades accordingly. At $s = 0.70$, the reconstruction is partially correct but no longer reliable for practical applications.

The relationship between masking severity and reconstruction quality is approximately linear until the critical threshold of 50%, beyond which performance degrades rapidly.

5.3 Overall Conclusions

This laboratory successfully demonstrated two powerful applications of autoencoders: unsupervised dimensionality reduction for visualization and supervised image inpainting for reconstruction of corrupted data.

The **Task 2** demonstrates how extreme compression ($784 \rightarrow 2$) enables visualization but limits discriminative capacity and the **Task 3** moderate compression ($784 \rightarrow 64$) preserves information necessary for high-quality reconstruction

The contrasting architectures highlight that autoencoder design must match the application: minimal bottlenecks for visualization require accepting training difficulty, while reconstruction tasks demand sufficient model capacity.